INTEGRATION RECOVERY RUNBOOK

VC

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Operations Integration Trust Repair

Restoring vCenter and vSAN adapter collection in VCF Operations after the vCenter VMCA certificate was regenerated — certificate re-trust, adapter re-validation, and the vSAN adapter delete/re-add.

VCF OPERATIONS | VCENTER TRUST | CERTIFICATE | ADAPTER RECOVERY | VSAN ADAPTER

VCF 9.0 VMware Cloud Foundation

VCF Operations Integration Trust Repair After vCenter Certificate Regeneration

Prepared by: Virtual Control LLC **Date:** June 5, 2026 **Document Version:** 1.0 **Classification:** Internal **Platform:** VMware Cloud Foundation 9.0 — VCF Operations 9.0.1

> **Scope.** Diagnosis and repair of the VCF Operations vCenter (VMWARE) and vSAN adapter collection failures after the management vCenter's machine SSL / VMCA certificate was regenerated — certificate re-trust, adapter re-validation, and the vSAN adapter delete/re-add — plus correct active-alert counting. Every command is shown as executed. **Companion docs.** - [`SDDC-Manager-UI-Stuck-Initializing-vCenter-SSH-HostKey-Repair-KB316028-2026-06-05.md`](./SDDC-Manager-UI-Stuck-Initializing-vCenter-SSH-HostKey-Repair-KB316028-2026-06-05.md) — SDDC Manager repair for the same vCenter regeneration event

---

Table of Contents

1. Executive Summary
2. Environment Reference
3. Problem Statement
4. Root Cause Analysis
5. Phase 1 — Diagnosis
6. Phase 2 — Restore the vCenter Adapter
7. Phase 3 — Restore the vSAN Adapter
8. Verification
9. How to Read Active Alerts
10. Lessons Learned
11. Command Appendix
12. Document Information

---

1. Executive Summary

The vCenter Server certificate (machine SSL leaf, and the VMCA root that signs it) was regenerated on the management vCenter. VCF Operations remained pinned to the **previous** root CA, so the TLS handshake between the VCF Operations collector and vCenter began to fail. Two adapter instances went to a non-collecting state as a result:

- `vcenter.lab.local` (VMWARE adapter) — "*Unable to connect to VC*", 0 metrics.
- `vcenter.lab.local - vSAN` (VirtualAndPhysicalSANAdapter) — failed downstream of the vCenter adapter, ultimately reporting "*Corresponding VC adapter instance is not configured.*"

A related symptom was that an attempt to **rotate / refresh the system-managed service account** for the vCenter adapter returned `INTERNAL_SERVER_ERROR` — this was a *consequence* of the broken trust, not a separate fault. The service account itself was never invalid.

Resolution followed the Broadcom-prescribed path:

1. Import the new vCenter **VMCA root CA** into the VCF Operations trusted-certificate store.

2. Re-validate the `vcenter.lab.local` adapter (no service-account rotation required) — collection resumed.
3. The vSAN adapter would not re-bind on its own; **delete it (with related objects) and re-add it** from the VCF account, after which it bound to the now-healthy vCenter adapter and resumed collection.

Final state: vCenter adapter ~2,750 metrics, vSAN adapter ~418 metrics, all integrations collecting. Total operator-driven time approximately 60 minutes, single operator, no support engagement.

> **Key takeaways.** (1) The fix is a **certificate trust** operation, not a credential operation — do not rotate the service account. (2) The vSAN adapter binds to a healthy vCenter adapter; it does not self-heal and is not auto-recreated after deletion — it must be re-added. (3) The VCF Operations "active alerts" total is inflated by recently-canceled alerts; count by `status == ACTIVE`.

1.1 In Plain English (for newcomers)

Picture two appliances that need to talk to each other: **vCenter** (runs the virtual machines) and **VCF Operations** (the monitoring system that watches vCenter and shows graphs and alerts).

For them to talk securely, vCenter shows VCF Operations an **ID card** (a TLS certificate). VCF Operations keeps a list of **ID cards it trusts**. As long as vCenter's ID card is on that trusted list, the two talk fine.

What broke: vCenter was **issued a brand-new ID card** (the certificate was regenerated). VCF Operations had only memorized the **old** card, so when vCenter showed the new one, VCF Operations said "*I don't recognize you*" and refused to connect. On screen that looked like **"Unable to connect to VC"** and the monitoring data stopped.

The confusing part: people often assume "can't connect" means a wrong password. It wasn't. The password (the login) was perfectly fine — the problem was purely "*I don't trust your new ID card.*" We proved this by logging into vCenter directly with the same credentials and it worked.

The fix, in three plain steps:

1. **Tell VCF Operations to trust the new ID card.** We exported vCenter's new certificate and imported it into VCF Operations' trusted list.
2. **Re-test the vCenter connection.** Now that the card is trusted, the connection works again and the graphs come back.
3. **Rebuild the vSAN piece.** vSAN monitoring "rides along" on the vCenter connection. It was so confused by the outage that re-testing wasn't enough — we had to **delete it and add it back fresh**, after which it reconnected on its own.

That's the whole repair: it was a *trust* problem (an unrecognized ID card), not a *password* problem, and the cure was to teach the monitoring system to recognize the new card.

---

2. Environment Reference

Item	Value
VCF Operations node (suite-api / UI)	`vcf-ops` — `192.168.1.77`
Management vCenter	`vcenter.lab.local` — `192.168.1.69`
VCF Operations collector	`VCF Operations Collector-vcf-ops`
VCF account / domain	`VCF 9 → mgmt`

Item	Value
VCF Operations local admin	`admin` (LOCAL auth source) — password vault ref `<LAB_PW>`
vCenter SSO administrator	`administrator@vsphere.local` — password vault ref `<vc-pw>`
Working directory for cert artifacts	`C:\VCF-Depot\vcenter-cert\`

Certificate facts captured during this incident:

Certificate	Subject	Validity (notBefore)	SHA-256 thumbprint
vCenter leaf (machine SSL)	`CN=vcenter.lab.local, C=US`	May 13 2026	`3C:4E:9B:ED:CE:B6:EA:5B:15:57:2D:78:0F:85:87:73:80:74:E8:49:68:F5:76:AE:F3:BA:E1:C9:2C:BD:68:5F`
VMCA root CA (signs the leaf)	`CN=CA, DC=vsphere, DC=local`	May 10 2026	`6D:0D:C1:6E:C7:C1:7C:E2:E1:C5:4D:E9:FD:90:1C:0A:4B:67:2C:54:4F:AD:30:A8:9D:98:37:4E:AF:1D:91:55`

> Thumbprints are environment-specific. Always re-capture them live (Section 5.2 / 6.1) before importing — never reuse the values above as-is.

---

3. Problem Statement

3.1 Symptoms

- VCF Operations → Integrations → Accounts showed `VCF 9` and the `mgmt` domain in **Warning**.
- `vcenter.lab.local` adapter: **0 metrics**, message "*Unable to connect to VC*".
- `vcenter.lab.local - vSAN` adapter: **0 metrics**, message progressing through "*Cannot parse vSAN SDK version*" → "*Error during collection*" → "*Corresponding VC adapter instance is not configured*".
- Editing the vCenter adapter and clicking **Validate Connection** returned "*Error trying to establish connection.*"
- Attempting to refresh the system-managed service account returned: `Failed to refresh service account <uuid> for vcenter.lab.local-...-ops. Error: message=Received service account with status: INTERNAL_SERVER_ERROR, code=0`.

3.2 What was NOT the cause

- vCenter was **healthy and reachable** throughout (HTTPS 200 on `/` and `/ui/`, REST returning 401 = service alive).
- The administrator credential was **valid** (a direct session POST returned 201).
- The service account did **not** require rotation; the rotate attempt only failed because trust was already broken.

---

4. Root Cause Analysis

> **Root cause.** The vCenter VMCA root CA (and the machine SSL leaf it signs) were regenerated. VCF Operations still trusted the prior root CA. With the new leaf presenting a chain that terminates in an untrusted root, the collector's TLS handshake to vCenter failed (PKIX path-building failure), surfacing as "*Unable to connect to VC.*"

Every dependent operation — metric collection, service-account refresh, and the vSAN adapter's bind to the vCenter adapter — failed as a direct consequence.

Why "Error trying to establish connection" rather than an auth error. That message is a transport-layer failure (the TLS session could not be established), not a credential rejection. It is the tell that distinguishes a certificate/trust problem from a username/password problem. An auth fault instead reports "*Cannot complete login due to an incorrect user name or password.*"

Why the vSAN adapter failed. The VirtualAndPhysicalSANAdapter does not connect to vCenter independently for inventory association — it binds to a vCenter (VMWARE) adapter instance that monitors the same vCenter on the same collector. With the vCenter adapter down, the vSAN adapter had no healthy parent to bind to, hence "*Corresponding VC adapter instance is not configured.*"

Supporting references.

- Broadcom KB 421016 — *vCenter Integration / data collection failure in VCF Operations 9.0 with SSLHandshakeException* (cert replaced → PKIX path build failure → "Unable to connect to VC"; fix = import the cert into the trusted store and re-validate).
- Broadcom KB 385033 — *Certificate replacement for vCenter fails at re-trust* (manual re-trust; no service-account rotation).
- VCF Operations 9.0 Known Issues — *Following the redeployment of a workload domain, the vCenter/vSAN adapter enters a Warning state.* Workaround: delete the affected domain adapter and re-add it from the VCF adapter's Domains view. A VMCA regeneration produces the same trust break as a redeploy, so the same workaround applies to the vSAN child.
- Broadcom TechDocs — *Configure a vSAN Adapter Instance*: a corresponding vCenter adapter monitoring the same vCenter Server must exist on the same node.

---

5. Phase 1 — Diagnosis

All commands below were run from the operator workstation (Git-Bash / OpenSSL / curl / Python available). Passwords are shown as vault references; substitute the live secret at run time.

5.1 Confirm vCenter is actually up

```
 # TCP 443 reachable?
(echo > /dev/tcp/192.168.1.69/443) >/dev/null 2>&1 && echo "443 OPEN" || echo "443 unreachable"

# HTTP status of UI and health endpoints
curl -k -s -o /dev/null -w "/        -> %{http_code}\n" --max-time 10 https://192.168.1.69/
curl -k -s -o /dev/null -w "/ui/     -> %{http_code}\n" --max-time 10 https://192.168.1.69/ui/
curl -k -s -o /dev/null -w "/api/appliance/health/system -> %{http_code}\n" \
  --max-time 10 https://192.168.1.69/api/appliance/health/system
```

Expected: `443 OPEN`, `/` and `/ui/` = `200`, health endpoint = `401` (service alive, auth-gated). This proves the problem is not vCenter availability.

5.2 Inspect the certificate vCenter is presenting

```
echo | openssl s_client -connect 192.168.1.69:443 -servername vcenter.lab.local 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -fingerprint -sha256
```

Read the `notBefore` date. If it is more recent than the last known-good integration, the certificate was regenerated — this is the smoking gun.

5.3 Confirm the administrator credential is valid

```
 # 201 = credential good; 401 = credential bad
curl -k -s -o /dev/null -w "POST /api/session -> %{http_code}\n" --max-time 20 \
  -u "administrator@vsphere.local:<vc-pw>" -X POST "https://192.168.1.69/api/session"
```

A `201` here, combined with a transport error in the UI, confirms the fault is **trust**, not credentials.

5.4 Read adapter state from the VCF Operations suite-api

Acquire a token, then list adapters. (The reusable parser `parse_adapters.py` is in Section 11.4.)

```
 OPS=192.168.1.77
TOK=$(curl -k -s --max-time 20 -X POST "https://$OPS/suite-api/api/auth/token/acquire" \
  -H "Content-Type: application/json" -H "Accept: application/json" \
  -d '{"username":"admin","authSource":"LOCAL","password":"<LAB_PW>"}' \
  | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')

curl -k -s --max-time 30 -H "Authorization: vRealizeOpsToken $TOK" -H "Accept: application/json" \
  "https://$OPS/suite-api/api/adapters" | python parse_adapters.py
```

> The `messageFromAdapterInstance` field **lags** — it reflects the last completed cycle, not live state. Trust `numberOfMetricsCollected` for whether collection is actually happening.

---

6. Phase 2 — Restore the vCenter Adapter

6.1 Export the new certificate material

vCenter presents only the leaf, so the **VMCA root CA** must be obtained separately (it is what the trusted store needs to validate the chain). The root also covers the vSAN adapter cert and any future leaf renewal.

```
 OUT="C:/VCF-Depot/vcenter-cert"; mkdir -p "$OUT"

# (Optional) capture the presented chain for the record
echo | openssl s_client -connect 192.168.1.69:443 -servername vcenter.lab.local -showcerts \
  2>/dev/null > "$OUT/raw.txt"

# Download the VMCA trusted-root bundle published by vCenter
curl -k -s --max-time 30 -o "$OUT/download.zip" "https://192.168.1.69/certs/download.zip"
cd "$OUT" && unzip -o download.zip      # extracts certs/{lin,mac,win}/<hash>.0[.crt]

# Identify and verify the root CA (the win/<hash>.0.crt is PEM)
openssl x509 -in certs/win/*.0.crt -noout -subject -issuer -dates -fingerprint -sha256

# Stage a clearly-named copy for import
cp certs/win/*.0.crt vmca-root-ca.crt
```

Confirm the staged file is a self-signed root (subject == issuer == `CN=CA, DC=vsphere, DC=local`) and note its SHA-256 thumbprint for the import confirmation dialog.

6.2 Import the root CA into the VCF Operations trusted store

VCF Operations UI:

1. **Administration → Control Panel → Trusted Certificates → Import.**
2. Upload `C:\VCF-Depot\vcenter-cert\vmca-root-ca.crt`.
3. Verify the displayed thumbprint matches the one from 6.1, then complete the import.

6.3 Re-validate the vCenter adapter

1. **Administration → Integrations → Accounts → VMware Cloud Foundation →** `VCF 9` **→** `mgmt` **→** `vcenter.lab.local` **→ Edit.**
2. Leave the existing **System Managed Credential** in place — **do not** rotate it and **do not** switch to a manual credential.
3. **Validate Connection.** With the root now trusted, validation succeeds (the earlier "*Error trying to establish connection*" is gone).
4. **Save.**

> **Result.** The `vcenter.lab.local` adapter clears its error and resumes collection on the next cycle (~2,750 metrics in this environment). No service-account rotation was performed or needed.

6.4 Watch collection resume

```
 # Re-run the Section 5.4 token + adapters call; numberOfMetricsCollected for
# vcenter.lab.local transitions 0 -> >0 within one collection cycle (~5 min).
```

---

7. Phase 3 — Restore the vSAN Adapter

After the vCenter adapter was healthy, the vSAN adapter still reported "*Corresponding VC adapter instance is not configured.*" The following were attempted in order.

7.1 What did NOT work (recorded so it is not retried)

- **Waiting** — 8+ collection cycles (~8 min) with the vCenter adapter healthy; the vSAN adapter never bound. This message is a configuration-association state, not a transient delay.
- **Edit → Validate → Save** on the vSAN adapter — validation passed but the binding did not re-establish.
- **Stop Collecting → Start Collecting** on the vSAN adapter — re-ran the same binding check; still unbound after 5 cycles.

7.2 The fix — delete and re-add the vSAN adapter

1. On the `vcenter.lab.local - vSAN` row → ⋮ → **Delete**.
2. In the delete dialog, **also select "Delete related objects."** The vSAN adapter's objects (vSAN World, the vSAN cluster object, disk groups, cache/capacity disks) are a separate resource set from the vCenter adapter's objects (VMs, hosts, datastores), so this does **not** affect the healthy vCenter adapter. Removing them clears the stale binding state.
3. Confirm the row disappears. (Note: the VCF adapter does **not** auto-recreate a manually-deleted vSAN child — verified over ~6 minutes — so it must be re-added explicitly.)
4. Re-add the vSAN adapter for the `mgmt` domain from the **VCF 9** account configuration; save the adapter instance.

Result. The re-added vSAN adapter binds to the now-healthy vCenter adapter (message field blank — no "not configured" error) and begins collecting (~418 metrics in this environment) within one to two cycles.

7.3 Watch the vSAN adapter bind

```
 # Re-run the Section 5.4 call; vcenter.lab.local - vSAN goes
# absent -> metrics=None (initializing) -> metrics=0 -> metrics>0.
# A blank message field (not "Corresponding VC adapter instance is not configured")
# is the confirmation that the bind succeeded.
```

---

8. Verification

```
 OPS=192.168.1.77
TOK=$(curl -k -s --max-time 20 -X POST "https://$OPS/suite-api/api/auth/token/acquire" \
  -H "Content-Type: application/json" -H "Accept: application/json" \
  -d '{"username":"admin","authSource":"LOCAL","password":"<LAB_PW>"}' \
  | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')

curl -k -s --max-time 30 -H "Authorization: vRealizeOpsToken $TOK" -H "Accept: application/json" \
  "https://$OPS/suite-api/api/adapters" | python parse_adapters.py
```

Healthy end state (this environment):

Adapter	Metrics	State
`vcenter.lab.local`	~2,750	Collecting
`vcenter.lab.local - vSAN`	~418	Collecting
`nsx-manager`	~2,445	Collecting
`VCF Operations Adapter (vcf-ops)`	~3,390	Collecting
`Infrastructure Health Adapter`	~127	Collecting
`Infrastructure Management Adapter`	2	Collecting

In the UI, `VCF 9` and the `mgmt` domain return from **Warning** to **green/Collecting** once the vSAN child is healthy.

---

9. How to Read Active Alerts

The "active alerts" total is misleading. The suite-api `activeOnly=true` query (and the matching UI badge) returns recently-**canceled** alerts as well, so the headline number is inflated. During this incident the query returned ~1,016, of which only **8 were genuinely ACTIVE** — the remaining ~1,008 were CANCELED and aging out. Always count by `status == ACTIVE`.

9.1 UI

VCF Operations → **Alerts** (or **Troubleshoot** → **Alerts**) → filter **Status = Active**. Do not rely on the raw count badge.

9.2 API — total in the activeOnly view (inflated; for reference only)

```
curl -k -s --max-time 30 -H "Authorization: vRealizeOpsToken $TOK" -H "Accept: application/json" \
  "https://$OPS/suite-api/api/alerts?activeOnly=true&pageSize=1" \
  | python -c 'import sys,json; print(json.load(sys.stdin)["pageInfo"]["totalCount"])'
```

9.3 API — the correct count (tally by status client-side)

```
curl -k -s --max-time 60 -H "Authorization: vRealizeOpsToken $TOK" -H "Accept: application/json" \
  "https://$OPS/suite-api/api/alerts?activeOnly=true&pageSize=2000" | python -c '
import sys,json,collections
d=json.load(sys.stdin)
c=collections.Counter(a.get("status") for a in d.get("alerts",[]))
print(dict(c))                                  # e.g. {"CANCELED":1008,"ACTIVE":8}
print("TRULY ACTIVE:", c.get("ACTIVE",0))
'
```

> The per-criticality query parameter (`alertCriticality=<LEVEL>`) was observed to be **ignored** by this build — it returned the full total for every level. Tally `alertLevel` client-side instead, exactly as above.

9.4 API — list the genuinely active alerts

```
curl -k -s --max-time 60 -H "Authorization: vRealizeOpsToken $TOK" -H "Accept: application/json" \
  "https://$OPS/suite-api/api/alerts?activeOnly=true&pageSize=2000" | python -c '
import sys,json
d=json.load(sys.stdin)
for a in d.get("alerts",[]):
    if a.get("status")=="ACTIVE":
        print(a.get("alertLevel"), "|", a.get("alertDefinitionName"), "|", a.get("resourceId"))
'
```

9.5 What "dropping" means

The CANCELED entries purge from the activeOnly result on their own over the following hours; that is the count visibly falling. It does **not** indicate live problems being resolved. The number to track is the ACTIVE tally from 9.3.

9.6 In Plain English (for newcomers)

Think of VCF Operations as a security guard who writes an incident report — an **alert** — every time something looks off. When the problem goes away, the guard does not shred the old report. He stamps it **"CANCELED"** and leaves it in the tray for a while before filing it away for good.

The "active alerts" number on the dashboard counts **everything still in that tray** — both the brand-new reports *and* the stamped-CANCELED ones that have not been filed yet. That is why the number looked alarming (about 1,000). Almost all of those were already-resolved reports waiting to be cleared out.

The number you actually care about is how many reports are **still happening right now** — the ones *not* stamped canceled. The system calls those **ACTIVE**. There were only **8**, and they were ordinary housekeeping items (a license reminder, a busy VM, a vSAN capacity note) — nothing to do with the outage we just fixed.

So two simple rules:

- **A big "active alerts" number is not automatically bad.** Open the **Alerts** page and set the filter to **Status = Active**. Read *that* number.

- **The big number shrinking over a few hours is normal.** That is just the old canceled reports being filed away — not a sign that new problems are being solved.

---

10. Lessons Learned

- **L1 — Transport error vs auth error.** "*Error trying to establish connection*" / "*Unable to connect to VC*" is a TLS/trust failure; "*Cannot complete login due to an incorrect user name or password*" is a credential failure. Diagnose them differently — the first is a certificate import, the second is a credential update.
- **L2 — A service-account refresh that returns `INTERNAL_SERVER_ERROR` after a cert change is a symptom, not the disease.** Fix trust first; do not rotate the account. Rotation cannot succeed while the collector cannot open a trusted session.
- **L3 — Import the VMCA root, not the leaf.** vCenter presents only the leaf; the trusted store needs the root to validate the chain, and the root also covers the vSAN cert and future leaf renewals. Source it from `https://<vc>/certs/download.zip`.
- **L4 — The vSAN adapter binds to the vCenter adapter.** It has no independent inventory connection. Fix the vCenter adapter before touching vSAN, and expect "*Corresponding VC adapter instance is not configured*" until the parent is healthy.
- **L5 — The vSAN adapter does not self-heal and is not auto-recreated.** Waiting, re-validating, and stop/start do not re-establish a broken binding. Delete it (with related objects) and re-add it. Deleting related objects is safe for the vCenter adapter because the resource sets are disjoint.
- **L6 — Adapter `messageFromAdapterInstance` lags.** Judge success by `numberOfMetricsCollected`, not by the message text.
- **L7 — Count alerts by `status == ACTIVE`.** The `activeOnly=true` total and the per-criticality filter are both unreliable in this build.

---

11. Command Appendix

11.1 Diagnosis (vCenter + credential)

```
 (echo > /dev/tcp/192.168.1.69/443) >/dev/null 2>&1 && echo "443 OPEN" || echo "443 unreachable"
curl -k -s -o /dev/null -w "/ -> %{http_code}\n"    --max-time 10 https://192.168.1.69/
curl -k -s -o /dev/null -w "/ui/ -> %{http_code}\n" --max-time 10 https://192.168.1.69/ui/
curl -k -s -o /dev/null -w "health -> %{http_code}\n" --max-time 10 \
  https://192.168.1.69/api/appliance/health/system
echo | openssl s_client -connect 192.168.1.69:443 -servername vcenter.lab.local 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -fingerprint -sha256
curl -k -s -o /dev/null -w "session -> %{http_code}\n" --max-time 20 \
  -u "administrator@vsphere.local:<vc-pw>" -X POST "https://192.168.1.69/api/session"
```

11.2 Certificate export

```
 OUT="C:/VCF-Depot/vcenter-cert"; mkdir -p "$OUT"; cd "$OUT"
echo | openssl s_client -connect 192.168.1.69:443 -servername vcenter.lab.local -showcerts \
  2>/dev/null > raw.txt
curl -k -s --max-time 30 -o download.zip "https://192.168.1.69/certs/download.zip"
unzip -o download.zip
openssl x509 -in certs/win/*.0.crt -noout -subject -issuer -dates -fingerprint -sha256
cp certs/win/*.0.crt vmca-root-ca.crt
```

11.3 suite-api token + status + alerts

```
OPS=192.168.1.77
TOK=$(curl -k -s --max-time 20 -X POST "https://$OPS/suite-api/api/auth/token/acquire" \
  -H "Content-Type: application/json" -H "Accept: application/json" \
  -d '{"username":"admin","authSource":"LOCAL","password":"<LAB_PW>"}' \
  | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')

# adapters
curl -k -s -H "Authorization: vRealizeOpsToken $TOK" -H "Accept: application/json" \
  "https://$OPS/suite-api/api/adapters" | python parse_adapters.py

# alerts: inflated total
curl -k -s -H "Authorization: vRealizeOpsToken $TOK" -H "Accept: application/json" \
  "https://$OPS/suite-api/api/alerts?activeOnly=true&pageSize=1" \
  | python -c 'import sys,json; print(json.load(sys.stdin)["pageInfo"]["totalCount"])'

# alerts: correct active count + list
curl -k -s -H "Authorization: vRealizeOpsToken $TOK" -H "Accept: application/json" \
  "https://$OPS/suite-api/api/alerts?activeOnly=true&pageSize=2000" | python -c '
import sys,json,collections
d=json.load(sys.stdin)
print("by status:", dict(collections.Counter(a.get("status") for a in d.get("alerts",[]))))
for a in d.get("alerts",[]):
    if a.get("status")=="ACTIVE":
        print(a.get("alertLevel"),"|",a.get("alertDefinitionName"),"|",a.get("resourceId"))
'
```

11.4 `parse_adapters.py` (adapter status helper)

```
import sys, json
d = json.load(sys.stdin)
for a in d.get("adapterInstancesInfoDto", []):
    name = a.get("resourceKey", {}).get("name", "")
    if name in ("vcenter.lab.local", "vcenter.lab.local - vSAN"):
        m = a.get("numberOfMetricsCollected")
        msg = a.get("messageFromAdapterInstance", "")
        print("%s | metrics=%s | %s" % (name, m, msg))
```

> To list every adapter (not just the two), drop the `if name in (...)` filter and print `name`, `adapterKindKey`, `numberOfMetricsCollected`, and `messageFromAdapterInstance`.

11.5 UI action summary

Step	UI path
Import root CA	Administration → Control Panel → Trusted Certificates → Import
Re-validate vCenter	Administration → Integrations → Accounts → VMware Cloud Foundation → VCF 9 → mgmt → vcenter.lab.local → Edit → Validate Connection → Save
Delete vSAN adapter	vcenter.lab.local - vSAN row → ⋮ → Delete → check "Delete related objects"
Re-add vSAN adapter	VCF 9 account configuration → mgmt domain → re-add vSAN → Save adapter instance
View active alerts	Alerts (Troubleshoot → Alerts) → filter Status = Active

---

12. Document Information

Field	Value
Title	VCF Operations Integration Trust Repair After vCenter Certificate Regeneration
Document ID	VC-RUNBOOK-VCFOPS-TRUSTREPAIR-20260605
Version	1.0
Issued	2026-06-05
Author	Virtual Control LLC
Classification	Internal
Platform	VMware Cloud Foundation 9.0 / VCF Operations 9.0.1
Applies to	VCF Operations vCenter (VMWARE) and vSAN (VirtualAndPhysicalSANAdapter) adapter instances after a vCenter machine-SSL / VMCA certificate change
Primary references	Broadcom KB 421016; Broadcom KB 385033; VCF Operations 9.0 Known Issues; Broadcom TechDocs — Configure a vSAN Adapter Instance
Related Documents	SDDC Manager UI Stuck on "VMware Cloud Foundation is Initializing" — vCenter SSH Host-Key Repair (KB 316028) (2026-06-05)
Distribution	Lab notebook

Revision History

Version	Date	Notes
1.0	2026-06-05	Initial issue. Documents diagnosis, certificate re-trust, vCenter adapter re-validation, vSAN adapter delete/re-add, verification, and corrected active-alert counting, with the full command set as executed.

Document End

See also: - SDDC Manager UI Stuck on "VMware Cloud Foundation is Initializing" — vCenter SSH Host-Key Repair (KB 316028) (2026-06-05) — parallel SDDC Manager fix for the same vCenter regeneration event - Broadcom KB 421016 — *vCenter data collection failure in VCF Operations 9.0 with SSLHandshakeException*